

Lesson 4: Flexbox for Modern Layouts

Mission Brief

Every good crew needs formation — ships lined up side by side, or stacked in a column for a tight corridor. Flexbox is the tool that lets you arrange your page's elements into flexible rows and columns that adapt automatically. Today, Navigator, you'll take manual command of layout direction, alignment, and spacing.

Learning Objectives

- Enable Flexbox with `display: flex` and identify the main axis vs. cross axis
- Control direction with `flex-direction`
- Align and distribute items with `justify-content` and `align-items`
- Control wrapping with `flex-wrap`
- Use `gap` and individual item sizing (`flex-grow` , `flex-shrink` , `flex-basis`)

What You'll Build

A responsive navigation bar and a "Skills" row of evenly spaced cards, both built with Flexbox, added to your Crew Profile page.

Prerequisites

Lesson 3: linking a stylesheet, box model, class selectors.

Key Concepts

- Flex container vs. flex item
- Main axis vs. cross axis
- `flex-direction`
- `justify-content`
- `align-items`
- `flex-wrap`
- `gap`
- `flex: grow shrink basis`

Concept Explanation

Before Flexbox, arranging elements side by side required hacks like `float` or `display: inline-block`, which were fragile and unintuitive. Flexbox (Flexible Box Layout) solves this with a simple mental model: you designate one element as a **flex container** by setting `display: flex` on it, and every direct child automatically becomes a **flex item** that Flexbox arranges along a single axis.

That axis is called the **main axis**, and its direction is set by `flex-direction: row` (the default, left to right) or `column` (top to bottom). Perpendicular to the main axis is the **cross axis**. This distinction matters because the two core alignment properties each control a different axis: `justify-content` positions items along the **main axis** (e.g. `flex-start`, `center`, `space-between`, `space-around`), while `align-items` positions items along the **cross axis** (e.g. `flex-start`, `center`, `stretch`).

By default, flex items try to fit on one line, shrinking if necessary. `flex-wrap: wrap` allows items to flow onto multiple lines instead of squeezing — essential for responsive designs where a row of cards should become two or three rows on a narrow screen.

Spacing between flex items used to require margin tricks; the modern `gap` property (e.g. `gap: 16px`) adds consistent spacing between items without affecting the outer edges — much cleaner.

Individual items can also be tuned. `flex-grow: 1` tells an item to expand and consume any leftover space in the container; `flex-shrink` controls how eagerly an item shrinks when space is tight; `flex-basis` sets a starting size before growing/shrinking is applied. The shorthand `flex: 1 1 200px;` means "grow: 1, shrink: 1, basis: 200px" — a very common pattern for equally-sized, responsive cards.

Flexbox is fundamentally **one-dimensional** — it excels at arranging items in a single row or column. When you need to control rows AND columns together (a true grid), that's what CSS Grid (next lesson) is for.

Guided Coding

```

/* Flexbox navigation bar */
nav ul {
  display: flex;
  flex-direction: row;
  justify-content: flex-end; /* push nav links to the right */
  align-items: center;       /* vertically center them */
  list-style: none;
  gap: 24px;
  flex-wrap: wrap;          /* wrap to a new line on very small screens */
}

/* Flexbox card row for skills */
.skills-row {
  display: flex;
  flex-direction: row;
  flex-wrap: wrap;
  gap: 20px;
  justify-content: center;
}

.skill-card {
  background-color: #ffffff;
  border: 1px solid #dde3f0;
  border-radius: 10px;
  padding: 20px;
  flex: 1 1 200px; /* grow, shrink, start at 200px – cards share space evenly */
  max-width: 280px;
  text-align: center;
}

.skill-card h3 {
  color: #4361ee;
  margin-bottom: 8px;
}

```

```

<!-- HTML to pair with .skills-row -->
<section id="skills">
  <h2>Skills in Progress</h2>
  <div class="skills-row">
    <div class="skill-card">
      <h3>HTML</h3>
      <p>Semantic structure and accessible markup.</p>
    </div>
    <div class="skill-card">
      <h3>CSS</h3>
      <p>Layout, color, and typography systems.</p>
    </div>
    <div class="skill-card">
      <h3>Flexbox</h3>
      <p>Flexible, responsive one-dimensional layouts.</p>
    </div>
  </div>
</section>

```

Code Walkthrough

- `nav ul { display: flex; }` turns the list into a flex container; its `<li>` children automatically become flex items laid out in a row.
- `justify-content: flex-end` pushes all nav items toward the end of the main axis (the right, in a row layout).
- `align-items: center` vertically centers the nav items relative to each other along the cross axis.
- `.skills-row { display: flex; flex-wrap: wrap; gap: 20px; }` creates a horizontally flowing card row that wraps onto new lines when the container gets too narrow, with consistent 20px gaps.
- `.skill-card { flex: 1 1 200px; max-width: 280px; }` — each card starts around 200px wide, is allowed to grow and shrink to fill space, but is capped at 280px so cards don't get absurdly wide on large screens.

Student Mission

1. Convert your Crew Profile nav into a Flexbox row using the code above.
2. Build a `.skills-row` of at least 4 `.skill-card` elements describing skills you're learning.
3. Change `justify-content` to `space-between` and then to `center`, and note the visual difference in a comment.
4. Add `flex-direction: column` to a new container somewhere on your page (e.g. a stacked "About" info block) and observe how `justify-content` / `align-items` swap which axis they control.

Challenge

Build a Flexbox-based "footer" with three columns (About / Links / Contact) that sit side by side on wide screens but stack vertically on narrow ones, using only `flex-wrap` and `flex-basis` — no media queries yet (those come in Lesson 6).

Experiment/Extension

On `.skill-card`, change `flex: 1 1 200px` to `flex: 0 0 200px` (grow and shrink both off) and resize your browser window narrower. Note in a comment what changes — cards should now overflow or stay fixed-width rather than adapting.

Common Mistakes

- Setting `display: flex` on the item instead of the container (it must go on the parent).
- Confusing `justify-content` (main axis) with `align-items` (cross axis) — swapping them is the single most common Flexbox bug.
- Forgetting `flex-wrap: wrap`, causing items to shrink painfully small instead of moving to a new line.
- Using margin hacks for spacing instead of the simpler `gap` property.
- Setting a fixed `width` and `flex-grow` together without understanding `flex-basis` takes priority over `width` in flex layout.

Debugging Tips

- Chrome/Edge/Firefox DevTools all show a small "flex" badge next to elements with `display: flex` in the Elements panel — click it to get an interactive overlay showing the main axis, cross axis, and gaps visually.
- If items aren't lining up as expected, check the Computed tab for `flex-direction` — it's easy to forget the container is `column` from an earlier rule.
- Toggle `flex-wrap` on/off live in DevTools' Styles pane to instantly see wrapping behavior without editing your file.

Check Your Understanding

1. What CSS property turns a container into a flex container?
2. What is the difference between the main axis and the cross axis?
3. Which property controls spacing along the main axis: `justify-content` or `align-items`?
4. What does `flex-wrap: wrap` do?
5. What does the shorthand `flex: 1 1 200px` mean?

Mini Quiz + Answer Key

Quiz 1. Which value of `flex-direction` stacks items top to bottom? a) row b) column c) wrap d) stack 2. True or False: `gap` works only in CSS Grid, not Flexbox. 3. Which property vertically centers flex items in a row layout? 4. What happens to flex items by default if they don't fit on one line and `flex-wrap` is not set? 5. What are the three values in the `flex` shorthand, in order?

Answer Key 1. b) column 2. False — `gap` works in both Flexbox and Grid. 3. `align-items: center` 4. They shrink to try to fit on one line (no wrapping). 5. `flex-grow`, `flex-shrink`, `flex-basis`

Lesson Recap

- Flexbox arranges items along one axis at a time (main and cross).
- `display: flex` on the parent activates it; children become flex items automatically.
- `justify-content` controls the main axis; `align-items` controls the cross axis.
- `flex-wrap` and `gap` are essential for responsive, evenly spaced layouts.
- `flex-grow` / `shrink` / `basis` fine-tune individual item sizing.

Homework

Rebuild your Lesson 3 styled page's `<nav>` and skills section entirely with Flexbox as shown, and write a short comment block at the top of your CSS file summarizing, in your own words, when you'd reach for Flexbox.

Portfolio Project Connection

Flexbox built here directly powers the responsive **nav bar in all three projects**, the **hero section layout in Project 01**, and the **header/nav plus mobile hamburger menu in Project 02 (Responsive portfolio layout)**.